Weak-Memory Local Reasoning, Revisited

Ian Wehrman

Draft of November 12, 2026

Abstract

My 2012 dissertation proposed a program logic for local reasoning about concurrent programs on an x86-like, total-store-order memory model. It left two components unfinished: the soundness proof, and a proof system for assertions. This report returns to both with a mechanization in Rocq. The soundness theorem does not hold as printed: four of the printed rules derive false specifications. A repaired calculus is sound, with operational adequacy, and keeps the assertion language, the unit law for **emp**, the temporal frame rule and the printed load and store axioms. The report explains what was wrong, what had to change and why, and what the repaired logic can prove.

1 Introduction

The dissertation [1] set out to give a local reasoning principle for programs running on an x86-like machine, where each processor buffers its writes and other processors cannot see a write until it leaves the buffer. Its logic has three separating conjunctions. Spatial conjunction $P * Q$ splits memory by address, as in separation logic. Temporal conjunction $P \lhd Q$ splits it by time: P describes earlier writes and Q later ones, possibly to the same addresses. Strong temporal conjunction $P \blacktriangleleft Q$ requires both. Each has a frame rule, and a concurrency rule and a shared invariant in the style of concurrent separation logic sit on top.

The dissertation was candid about where it stopped. Its statement of contributions lists two major components as unfinished and admits a third gap:

1. there is no proof system for the assertion language;
2. the soundness proof for the specification logic is incomplete; and
3. the document "gives little indication of how proofs ought to proceed in the logic."

It also hypothesized that the logic was sound and that the remaining proofs were "mostly straightforward inductions."

This report returns to those three items, fourteen years later, with a mechanization in the Rocq prover. Its findings are as follows.

The soundness theorem does not hold as printed. Several auxiliary claims are false, and each has a local repair (Section 3.1). More seriously, the rules themselves are inconsistent with any adequate semantics: from SKIP, CONS, INV and FRAME-TM alone one can derive that a skip changes the contents of a memory cell (Section 3.2). No choice of safety predicate can repair that.

The cause is general. An unrestricted temporal frame rule, a right unit for temporal conjunction, the rule of consequence, and any command that gives away ownership of a cell cannot coexist (Section 3.3). The observation is independent of the memory model. It applies to any logic that combines a non-cancellative sequential conjunction with ownership transfer.

A repaired calculus is sound. The repair replaces the invariant rule by lock and unlock rules that transfer the invariant, restricts the temporal frame rule to specifications that tolerate overlapping history, and adjusts four side conditions (Section 4). The repaired calculus is proved sound by induction on derivations, with an adequacy theorem for the dissertation's machine (Section 7). The proofs are checked by the Rocq kernel and use no axioms.

Most of the design survives. The machine, the assertion language and its interpretation of atoms, the unit laws for **emp**, overlapping temporal history, the temporal and spatial frame rules, the concurrency rule, and the printed load and store axioms are all retained. So is the dissertation's assertion algebra, with one family of exceptions: every printed law that rebrackets or exchanges through strong temporal conjunction fails, and with it the strong temporal frame rule (Appendix A). [*pending:* algebra table in the appendix]

The logic can be used. Section 8 derives a shared container in which a thread acquires the lock, removes a cell, releases the lock, writes the cell privately, and later returns it, while another processor's write stays buffered throughout. [*pending:* the Section 4.1 example; the larger derivation; the assertion proof system]

The report assumes a reader who knows the dissertation and has forgotten its details. Section 2 recalls the original model and calculus in the original notation. Sections 3 and 4 say what was wrong and what changed. Sections 5 to 8 present the repaired model, calculus, results and derivations. Proofs are not reproduced; every claim cites the name of its Rocq theorem, and the appendix gives fuller statements.

2 The original model and calculus

This section recalls Chapter 4 of the dissertation, in its notation. Nothing here is new. Page and figure numbers refer to the dissertation.

2.1 The machine

A *memory system* is a triple $\mu = (h, B, K)$: a heap $h : \mathbb{L} \rightharpoonup \mathbb{V}$, an array of write buffers $B : \mathbb{P} \to (\mathbb{L} \times \mathbb{V})$ list with the oldest write first, and a set $K \subseteq \mathbb{P}$ of *blocked* processors. A state pairs a stack s with a memory system. The lock is not a separate component: it is free when $K = \emptyset$ and held by i when $K = \mathbb{P} \setminus \{i\}$.

Primitive commands carry the identifier of the processor that executes them. Their behaviour on processor i is as follows.

- $[e] := e'_i$ appends a write to $B(i)$. The address must be allocated, in the heap or in $B(i)$. A blocked processor may still store.
- $x := [e]_i$ requires $i \notin K$ and returns the newest write to that address in $B(i)$, or else the heap value.
- fence_i, lock_i and unlock_i require $B(i) = \varepsilon$. lock_i additionally requires $K = \emptyset$ and sets $K = \mathbb{P} \setminus \{i\}$; unlock_i reverses this.
- A silent step $\underset{\tau}{\to}$ moves the oldest write of some unblocked processor from its buffer to the heap.

Commands are built from primitives by sequencing $c\,;c'$, choice $c + c'$, iteration c^* and parallel composition $c \parallel c'$. There is no allocation. While a processor holds the lock, no other processor can load or flush, but others may continue to store and to assign to registers.

2.2 Generalized memories and separation

A *generalized memory system* $\nu = (h, B, K, \Gamma)$ adds a *buffer-completeness* set $\Gamma \subseteq \mathbb{P}$, with $h = \emptyset$ whenever $\Gamma = \emptyset$. Membership $i \in \Gamma$ means that buffer i is complete: nothing may be placed before it.

Three operations combine generalized memories. Write $\nu_k = (h_k, B_k, K_k, \Gamma_k)$. All three take $K_0 \cup K_1$ and $\Gamma_0 \cup \Gamma_1$ and require $K_0 \cap K_1 = \emptyset$.

Spatial, $\nu_0 \mathbin{\widehat{*}} \nu_1$. The address domains of the two sides, counting both heaps and buffers, are disjoint. The heap is $h_0 \uplus h_1$ and each buffer is an interleaving of the two.

Temporal, $\nu_0 \mathbin{\widehat{\lhd}} \nu_1$. The heap is $h_0 \backslash\backslash h_1$, so later values win, and each buffer is $B_0(i) +\!\!+ B_1(i)$. The sides may share addresses, subject to a compatibility condition: an unblocked pending write in ν_0 may not target an address in the heap of ν_1, nor an address pending on a different unblocked processor in ν_1. Finally $B_0(i) = \varepsilon$ for every $i \in \Gamma_1$.

Spatiotemporal, $\nu_0 \mathbin{\widehat{\#}} \nu_1$. As temporal, but each buffer is any interleaving whose lookup function agrees with $B_0(i) +\!\!+ B_1(i)$, and the condition on Γ_1 is dropped. It contains both of the others and is used only in the semantics of specifications.

Strong temporal separation $\nu_0 \mathbin{\widehat{\blacktriangleleft}} \nu_1$ is $\nu_0 \mathbin{\widehat{\lhd}} \nu_1$ restricted to pairs that are also spatially compatible.

2.3 Assertions

Assertions are given by

$$\begin{aligned} P \ ::= \ & b \mid P \vee P \mid P \wedge P \mid \exists x : P \mid \forall x : P \mid \mathbf{emp} \mid \mathbf{bar}_e \mid \mathbf{lock}_e \mid e \rightsquigarrow_{e'} e'' \\ & \mid P * P \mid P \lhd P \mid P \mathrel{-\!\!*} P \mid P \mathrel{-\!\!\lhd} P \mid P \mathrel{\lhd\!\!-} P \end{aligned}$$

and are interpreted over a stack and a generalized memory. The atoms are:

- **emp**: empty heap, empty buffers, $K = \emptyset$; Γ is unconstrained.
- $\mathbf{bar}_i$: as **emp**, with $i \in \Gamma$ when i is a processor.
- $\mathbf{lock}_i$: empty heap and buffers, with $K = \mathbb{P} \setminus \{i\}$.
- $e \rightsquigarrow_i e'$ ("leads to"): either a single *pending* write (e, e') in $B(i)$ and nothing else, or a single *flushed* heap cell $e \mapsto e'$ with $i \in \Gamma$.

$P * Q$ and $P \lhd Q$ hold of ν when ν is a spatial, respectively temporal, combination of a model of P and a model of Q. The three implications are the corresponding adjoints. Two abbreviations matter below:

$$P \blacktriangleleft Q =_{df} (P * Q) \wedge (P \lhd Q) \qquad\qquad e \mapsto e' =_{df} (\exists x : e \rightsquigarrow_x e') \lhd (\forall x : \mathbf{bar}_x).$$

Points-to therefore describes a flushed cell with *every* processor buffer-complete. The dissertation claims that every assertion is closed under flushing (Proposition 9), and that each conjunction is associative with **emp** as a unit (Figure 4.5).

2.4 Specifications and rules

A specification $J \vdash \{P\}\ c\ \{Q\}$ has an invariant J for shared state and a private precondition and postcondition. Figure 1 reproduces the axioms and rules.

Three features of the design are worth recalling, because the repair turns on them. First, LOCK and UNLOCK produce and consume only the token $\mathbf{lock}_i$. The invariant reaches the lock holder through INV, which may be applied to any command executed while the lock is held and allows the private state to exchange cells with J. Second, FRAME-TM has only the usual variable condition: any history R may be placed before any specification. Third, temporal conjunction is not cancellative, as the dissertation remarks: $\mu \mathbin{\widetilde{\lhd}} \mu_0 = \mu \mathbin{\widetilde{\lhd}} \mu_1$ does not imply $\mu_0 = \mu_1$.

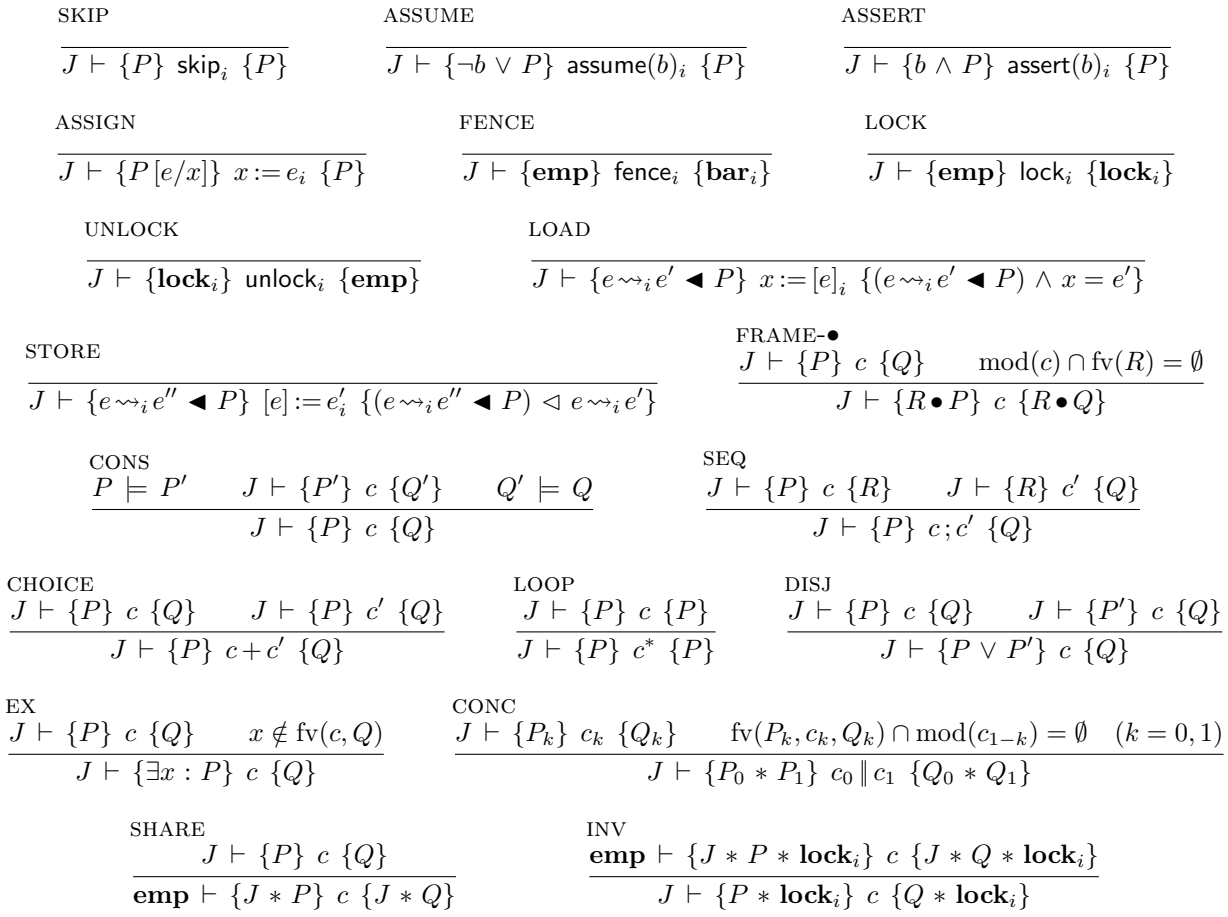


Figure 1: The printed axioms and rules (Figures 4.7 and 4.8). FRAME-• stands for three rules, FRAME-SP, FRAME-TM and FRAME-STM, with • read as $*$, $\lhd$ and $\blacktriangleleft$.

2.5 Semantics and status

Truth of a specification is defined through a family of predicates $safe_n(c, s, \mu, J, Q)$ after Vafeiadis. The predicate holds at $n+1$ when: if c is skip then $(s, \mu, \mathbb{P}) \models Q$; and for every μ_J, μ_F and $\mu_0 \in \mu_J \mathbin{\widetilde{*}} (\mu_F \mathbin{\widetilde{\#}} \mu)$ that is lock-complete, with either $(s, \mu_J, \mathbb{P}) \models J$ or the lock held in μ_0, the configuration does not abort, and every step to (c', s', μ_1) admits a new decomposition $\mu_1 \in \mu'_J \mathbin{\widetilde{*}} (\mu'_F \mathbin{\widetilde{\#}} \mu')$ with $\mu'_F \preceq \mu_F$, the same condition on J, and $safe_n(c', s', \mu', J, Q)$. A specification is true, $J \models \{P\}\ c\ \{Q\}$, when $(s, \mu, \mathbb{P}) \models P$ implies $safe_n(c, s, \mu, J, Q)$ for all n. Local states are always read at full completeness, $\Gamma = \mathbb{P}$.

Theorem 1 states that provable specifications are true. The dissertation gives soundness lemmas for the axioms in an appendix, sketches a frame lemma for the spatiotemporal operator $\widetilde{\#}$, and reduces the spatial frame rule to Conjecture 1, which is left open. Associativity of buffer overriding (Proposition 2) and of the three separations, and flushing closure of assertions (Proposition 9), are stated with at most a sketch.

3 What was wrong

The problems are presented from the most local to the most structural, which is also the order in which the mechanization found them. Each is a theorem checked by Rocq against a transcription of the printed definitions; the theorem's name is given so that the statement can be inspected.

3.1 Local errors with local repairs

Table 1 lists six printed claims that are false as stated. None is deep, and each has a repair that leaves the design intact.

Printed claim	Counterexample	Repair
ASSIGN is sound for every invariant	With J the assertion $x = 0$, the axiom derives $J \vdash \{\mathbf{true}\}\ x := 1_1\ \{\mathbf{true}\}$, and the assignment falsifies J with the lock free. `printed_assignment_invalid`	Require $x \notin \mathrm{fv}(J)$.
Buffer overriding is associative (Prop. 2)	Three one-element buffers writing 1, 2, 3 to one address: the list $[2,1,3]$ is in $a \mathbin{\backslash\!\backslash} (b \mathbin{\backslash\!\backslash} c)$ but not in $(a \mathbin{\backslash\!\backslash} b) \mathbin{\backslash\!\backslash} c$. `printed_buffer_override_not_associative`	A write from the later buffer may be taken only after the earlier buffer's writes to that address.
Spatiotemporal separation is associative	Compatibility ignores blocked processors, so whether two memories compose can depend on a third that supplies the blocking fact. `printed_spatiotemporal_not_associative`	Check conflicts whether or not the processor is blocked.
Assertions are closed under flushing (Prop. 9)	$\mathbf{false} \mathrel{\triangleleft\!\!-} \mathbf{bar}_1$ holds vacuously while processor 1 has a pending write, and fails once the write flushes. `pointwise_temporal_wand_not_flush_closed`	Implications quantify over all futures of the current memory.
Conjecture 1	A cell duplicated across $\widetilde{\#}$ with equal values is not a spatial split. `printed_conjecture1_false`	Not needed by the final proof.
UNLOCK is sound for every invariant	With $J = \mathbf{false}$ and the lock held, unlock_i is enabled and nothing restores J. `arbitrary_invariant_unlock_invalid`	See Section 4.

Table 1: Printed claims that fail, each with a checked counterexample.

Two remarks. The repaired compatibility relation is stronger than the printed one, so it rejects some compositions the dissertation admits. Those are states in which a blocked processor has a pending write to a cell that someone else owns, which arise only under the "daring" rules that the dissertation proposed and did not adopt. And the repaired implications are the ordinary Kripke reading of implication over the flushing preorder; the three adjunctions between conjunctions and implications still hold.

The failure of UNLOCK points at something larger. The printed safety predicate asks for the invariant only while the lock is free. Once a thread holds the lock, the predicate has forgotten that J held at the moment of acquisition, so neither UNLOCK nor INV can be justified from it. That alone would call for a different predicate. The next subsection shows that it calls for different rules.

3.2 The printed rules are inconsistent

The derivations below use the dissertation's own points-to, $x \mapsto v$, and are checked against its own interpretation of assertions.

History can be invented. A model is a memory, not a record of how the memory came about. So a current cell is also a model of any stale value followed by that cell:

$$x \mapsto 7 * \mathbf{lock}_i \;\models\; x \mapsto 5 \lhd (x \mapsto 7 * \mathbf{lock}_i). \tag{1}$$

The stale cell is overridden by the later one, and nothing remains of it. This is the non-cancellativity that the dissertation notes in passing.

A skip that changes memory. Let J be $\mathbf{emp} \vee x \mapsto 7$. The following is a derivation in the printed calculus.

1. $\mathbf{emp} \vdash \{J * x \mapsto 7 * \mathbf{lock}_i\}\ \mathsf{skip}_i\ \{J * \mathbf{emp} * \mathbf{lock}_i\}$, by SKIP and CONS: J cannot be its second disjunct, since the cell would occur twice, and in the first disjunct the cell can be reassigned to J.
2. $J \vdash \{x \mapsto 7 * \mathbf{lock}_i\}\ \mathsf{skip}_i\ \{\mathbf{emp} * \mathbf{lock}_i\}$, by INV. The thread has given its cell to the invariant.
3. $J \vdash \{x \mapsto 5 \lhd (x \mapsto 7 * \mathbf{lock}_i)\}\ \mathsf{skip}_i\ \{x \mapsto 5 \lhd (\mathbf{emp} * \mathbf{lock}_i)\}$, by FRAME-TM.
4. $J \vdash \{x \mapsto 7 * \mathbf{lock}_i\}\ \mathsf{skip}_i\ \{x \mapsto 5 * \mathbf{lock}_i\}$, by CONS, using (1) on the left and the unit law for **emp** on the right.

The conclusion says that a thread holding the lock and a cell containing 7 may do nothing and then own that cell containing 5. In the state where processor 1 holds the lock and x contains 7, the precondition holds and the postcondition does not. (`printed_points_stale_value_unsound`; the mechanized statement parses INV's premise as $J * (P * \mathbf{lock}_i)$.)

A skip that drains another processor's buffer. Spatial conjunction takes the union of the completeness sets and does not require them to be disjoint, so an empty neighbour can copy a cell's marks: $x \mapsto 7 * \mathbf{emp} \models x \mapsto 7 * \mathbf{bar}_2$. With J the cell $x \mapsto 7$, SKIP, CONS and INV give $J \vdash \{\mathbf{emp} * \mathbf{lock}_1\}\ \mathsf{skip}_1\ \{\mathbf{bar}_2 * \mathbf{lock}_1\}$, and FRAME-TM with a pending write $R = y \rightsquigarrow_2 17$ gives

$$J \vdash \{R \lhd (\mathbf{emp} * \mathbf{lock}_1)\}\ \mathsf{skip}_1\ \{R \lhd (\mathbf{bar}_2 * \mathbf{lock}_1)\}.$$

When processor 1 holds the lock and processor 2's write to y is still buffered, the precondition holds. The postcondition places a barrier for processor 2 after its pending write, which that memory does not satisfy. (`printed_points_foreign_barrier_unsound`.)

The strong temporal frame rule does not escape. One might hope that FRAME-STM is safe, since its frame is also spatially separate. It is not, because $R \blacktriangleleft P$ abbreviates $(R * P) \wedge (R \lhd P)$ and the two conjuncts may be witnessed by different splits. Take $R = \mathbf{emp} \vee x \mapsto 5$. In the state above, the spatial conjunct of $R \blacktriangleleft (x \mapsto 7 * \mathbf{lock}_1)$ holds with R empty, and the temporal conjunct holds with R the stale cell. Applying FRAME-STM to step 2 yields a postcondition that the unchanged state cannot satisfy. (`printed_strong_frame_counterexample`; this derivation uses INV and FRAME-STM but not FRAME-TM.)

All three conclusions are also invalid for the printed predicate $\mathit{safe}_n()$, but that is the lesser point. They are false under the dissertation's informal reading of a specification, and under any semantics for which a true specification describes what the program does.

3.3 Why no semantics could repair it

The first derivation is an instance of a general conflict.

Proposition 1 (`trilemma`)**.** *Suppose a logic of triples has an unrestricted temporal frame rule, the rule of consequence, and the entailment* $R \lhd \mathbf{emp} \models R$*. Suppose* $\{P\}\ c\ \{\mathbf{emp}\}$ *is provable. Then* $\{R \lhd P\}\ c\ \{R\}$ *is provable for every* R*. Hence if some execution of* c *starts in a state satisfying* $R \lhd P$ *and ends in a state that does not satisfy* R*, the logic is unsound.*

The proof is two rule applications. The content is in the last sentence. A triple $\{P\}\ c\ \{\mathbf{emp}\}$ with P owning a cell is a *transfer-out contract*: after c, the cell belongs to someone else. Whenever temporal conjunction lets later values override earlier ones, R may describe an older state of the same cell, as the store axiom's own postcondition does. Framing R across the transfer then hands the thread a stale copy of a cell it no longer owns.

So four things cannot all be kept: the unrestricted temporal frame rule, the right-unit law, transfer-out contracts ending in **emp**, and overlapping temporal history. A sound logic must give one up, and the choice determines its character:

- *No transfer out.* Cells may be taken from the invariant and never returned. This is sound and useless for a shared container.
- *No right unit.* Let **emp** record that a cell was given away. Framing survives, and every postcondition that mentions **emp** changes meaning.
- *Transfer leaves a mark.* Return an explicit tombstone assertion instead of **emp**. The algebra survives; every later specification carries the tombstones.
- *Restrict the frame.* Forbid temporal frames that overlap the cells being given away. Everything else is kept.

The first two were built and abandoned during this work, for the reasons given. The repaired calculus takes the fourth, and by Proposition 1 that restriction cannot be weakened: every frame that overlaps a released cell yields a false specification.

The second derivation has a different cause and a different repair. The all-processor completeness of points-to, and more generally the completeness marks of whatever the invariant holds, should not pass to a thread that acquires the lock. A thread can vouch only for its own buffer.

3.4 The dissertation's own example

Section 4.1 of the dissertation proves a small message-passing program correct, using an invariant that mentions two registers and the derived rule ATOMIC. That proof does not carry over to the repaired calculus: ATOMIC, SHARE and auxiliary-variable elimination are not available, and the concurrency rule now forbids either thread from modifying a variable free in the invariant. This shows that the proof does not port, not that its conclusion is false. [*pending:* the repaired derivation of the same program, and the recovery of its original conclusion]

4 What changed

[*pending:* to be written]

5 The repaired model

[*pending:* to be written]

6 The repaired calculus

[*pending:* to be written]

7 Soundness and adequacy

[*pending:* to be written]

8 How proofs proceed

[*pending:* to be written]

9 The mechanization

[*pending:* to be written]

10 Later work

[*pending:* to be written]

11 What remains open

[*pending:* to be written]

References

[1] Ian Wehrman. *Weak-Memory Local Reasoning*. PhD dissertation, The University of Texas at Austin, December 2012.

A Statements in full

[*pending:* fuller statements of the principal definitions and theorems, with their Rocq names; the table of the dissertation's assertion laws]